Fully Connected Neural Net (FCNN)

Objective: Build FCNN models to handle different data types.

Part 1: Tabular Modelling (Frameworks Allowed)

Dataset: UCI Adult Census Income

Goal: Model the relationship between demographic features and income level using a standard library (PyTorch).

1. Problem Definition

The objective of this task is to build a Fully Connected Neural Network (FCNN) to model the relationship between demographic attributes and income level using the UCI Adult Census dataset. The task is formulated as a binary classification problem, where the model predicts whether an individual earns:

- **≤ 50K (Class 0)**
- **50K (Class 1)**

The goal is to analyze how feature preprocessing, particularly feature scaling, impacts convergence behavior and final model accuracy.

2. Dataset Description

The UCI Adult Census dataset consists of demographic and socio-economic attributes of individuals. The dataset contains both numerical and categorical features.

Numerical Features:

- Age
- fnlwgt
- education-num
- capital-gain
- capital-loss
- hours-per-week

Categorical Features:

- workclass
- education

- marital-status
- occupation
- relationship
- race
- sex
- native-country

The dataset also contains missing values represented as "?", which were treated as a separate category to avoid data loss.

The target variable income originally contained string labels (<=50K, >50K, including trailing "." in test data), which were converted into binary values (0 and 1).

```
--- Training Data Sample ---
   age         workclass  fnlwgt  education  education-num  \
0   39         state-gov   77516  bachelors             13
1   50  self-emp-not-inc   83311  bachelors             13
2   38           private  215646    hs-grad              9
3   53           private  234721       11th              7
4   28           private  338409  bachelors             13

       marital-status         occupation   relationship   race     sex  \
0       never-married       adm-clerical  not-in-family  white    male
1  married-civ-spouse    exec-managerial        husband  white    male
2            divorced  handlers-cleaners  not-in-family  white    male
3  married-civ-spouse  handlers-cleaners        husband  black    male
4  married-civ-spouse     prof-specialty           wife  black  female

   capital-gain  capital-loss  hours-per-week native-country  income
0          2174             0              40  united-states       0
1             0             0              13  united-states       0
2             0             0              40  united-states       0
3             0             0              40  united-states       0
4             0             0              40           cuba       0
```

```
--- Test Data Sample ---
   age  workclass  fnlwgt     education  education-num      marital-status  \
0   25    private  226802          11th              7       never-married
1   38    private   89814       hs-grad              9  married-civ-spouse
2   28  local-gov  336951    assoc-acdm             12  married-civ-spouse
3   44    private  160323  some-college             10  married-civ-spouse
4   18          ?  103497  some-college             10       never-married

          occupation relationship   race     sex  capital-gain  capital-loss  \
0  machine-op-inspct    own-child  black    male             0             0
1    farming-fishing      husband  white    male             0             0
2    protective-serv      husband  white    male             0             0
3  machine-op-inspct      husband  black    male          7688             0
4                  ?    own-child  white  female             0             0

   hours-per-week native-country  income
0              40  united-states       0
1              50  united-states       0
2              40  united-states       1
3              40  united-states       1
4              30  united-states       0
```

3. Methodology

3.1 Dataset Processing

Since neural networks require numerical inputs, the following preprocessing steps were applied:

1. **String Cleaning**: All categorical features were converted to lowercase and stripped of leading/trailing spaces to ensure consistency.
2. **Handling Missing Values**: The "?" entries were retained as valid categories instead of removing rows.
3. **Label Encoding**: The target variable was converted to binary format:
   - $\leq 50K \rightarrow 0$
   - $50K \rightarrow 1$
4. **One-Hot Encoding**: All categorical variables were transformed using one-hot encoding.
5. **Column Alignment**: feature dimensions.
6. **Feature Scaling (Experiment 2 Only)**: Min-Max scaling was applied to numerical features to normalize them into the range [0,1].

4. Model Architecture

A 3-layer Fully Connected Neural Network was implemented using PyTorch.

Architecture:
Input Layer → 128 neurons → 64 neurons → Output Layer (1 neuron)

- Hidden Layer Activation: ReLU
- Output Layer: Linear (used with BCEWithLogitsLoss)
- Loss Function: Binary Cross Entropy with Logits
- Optimizer: Adam
- Batch Size: 256
- Epochs: 20
- Device: GPU (CUDA-enabled)

5. Experimental Setup

Two experiments were conducted:

Experiment 1 – Raw Numerical Features

The model was trained without applying feature scaling.

Experiment 2 – Min-Max Scaled Features

Numerical features were scaled using MinMaxScaler before training.

6. Results

Experiment 1 – Raw Data

```
Epoch [1/20], Loss: 81.1102
Epoch [2/20], Loss: 45.8161
Epoch [3/20], Loss: 50.9506
Epoch [4/20], Loss: 81.9768
Epoch [5/20], Loss: 37.1642
Epoch [6/20], Loss: 26.4080
Epoch [7/20], Loss: 14.4407
Epoch [8/20], Loss: 24.6347
Epoch [9/20], Loss: 30.6981
Epoch [10/20], Loss: 13.8893
Epoch [11/20], Loss: 19.2187
Epoch [12/20], Loss: 13.2320
Epoch [13/20], Loss: 17.2194
Epoch [14/20], Loss: 7.0495
Epoch [15/20], Loss: 15.5758
Epoch [16/20], Loss: 10.9147
Epoch [17/20], Loss: 13.4217
Epoch [18/20], Loss: 6.0953
Epoch [19/20], Loss: 8.5078
Epoch [20/20], Loss: 7.1010
Test Accuracy (Raw Data): 0.7868681284933358
```

- Final Test Accuracy: **78.68%**
- Training loss showed fluctuations and instability.
- Convergence was irregular due to large magnitude differences among features.

Experiment 2 – Scaled Data

```
Epoch [1/20], Loss: 0.4232
Epoch [2/20], Loss: 0.3436
Epoch [3/20], Loss: 0.3328
Epoch [4/20], Loss: 0.3227
Epoch [5/20], Loss: 0.3189
Epoch [6/20], Loss: 0.3137
Epoch [7/20], Loss: 0.3119
Epoch [8/20], Loss: 0.3081
Epoch [9/20], Loss: 0.3038
Epoch [10/20], Loss: 0.3020
Epoch [11/20], Loss: 0.2989
Epoch [12/20], Loss: 0.2975
Epoch [13/20], Loss: 0.2958
Epoch [14/20], Loss: 0.2935
Epoch [15/20], Loss: 0.2933
Epoch [16/20], Loss: 0.2898
Epoch [17/20], Loss: 0.2875
Epoch [18/20], Loss: 0.2864
Epoch [19/20], Loss: 0.2841
Epoch [20/20], Loss: 0.2844
Test Accuracy (Scaled Data): 0.8434985565997175
```

- Final Test Accuracy: **84.35%**
- Training loss decreased smoothly and consistently.
- Convergence was significantly more stable.

Accuracy Comparison

Experiment	Test Accuracy
Raw Data	78.68%
Scaled Data	84.35%

Scaling resulted in an improvement of approximately **5.7%** in test accuracy.

7. Analysis

The experiment clearly demonstrates that Fully Connected Neural Networks are sensitive to feature scale. When numerical features have significantly different magnitudes, gradient updates become unstable, leading to fluctuating loss behavior.

Applying Min-Max scaling:

- Stabilized gradient updates

- Improved convergence behavior
- Increased generalization performance
- Reduced training instability

The smoother loss curve in the scaled experiment confirms that normalization helps ensure balanced feature contribution during optimization.

8. Conclusion

This study highlights the importance of proper feature preprocessing in tabular neural network models. While the model achieved acceptable performance without scaling, normalization significantly improved both convergence stability and final accuracy.

Therefore, feature scaling is strongly recommended when training Fully Connected Neural Networks on tabular datasets.

Part 2: Vision & Feature Interpretation (Frameworks Allowed)

Dataset: MNIST Handwritten Digits

Goal: Bridge the gap between pixels and patterns using PyTorch.

1. Problem Definition

The objective of this task is to understand how a Fully Connected Neural Network (FCNN) processes visual data and what kind of internal feature representations it learns. The MNIST handwritten digit dataset is used as a benchmark for multi-class classification (digits 0–9).

This part focuses on two key aspects:

1. **Visualizing the learned weights of the first hidden layer.**
2. **Evaluating whether spatial structure is important for an FCNN using a pixel shuffle experiment.**

The aim is to analyze whether the network truly understands spatial patterns or simply learns statistical relationships between pixels and labels.

2. Dataset Description & Preprocessing

The MNIST dataset consists of grayscale images of handwritten digits, each of size 28×28 pixels. The dataset provided in CSV format contains:

- 60,000 training samples
- 10,000 test samples
- 785 columns per sample (1 label + 784 pixel values)

Preprocessing Steps:

1. Separated the label column from pixel values.
2. Normalized pixel values from range [0, 255] to [0, 1].
3. Converted data into PyTorch tensors (float32 for features, long for labels).
4. Used DataLoader with batch size 256 for efficient GPU training.

No additional spatial transformations were applied since the FCNN expects flattened input vectors.

3. Model Architecture & Hyperparameter Choices

A 3-layer Fully Connected Neural Network was implemented in PyTorch.

Architecture:

Input (784) → Hidden Layer 1 (256) → Hidden Layer 2 (128) → Output (10)

- Activation Function: ReLU
- Loss Function: CrossEntropyLoss
- Optimizer: Adam
- Learning Rate: 0.001
- Batch Size: 256
- Epochs: 15
- Device: CUDA-enabled GPU

Justification of Architecture

- 256 neurons in first layer provide sufficient capacity to capture diverse visual patterns.
- 128 neurons in second layer compress learned representations.
- CrossEntropyLoss is suitable for multi-class classification.
- Adam optimizer ensures stable and efficient convergence.

4. Results – Standard MNIST

Training on Standard MNIST

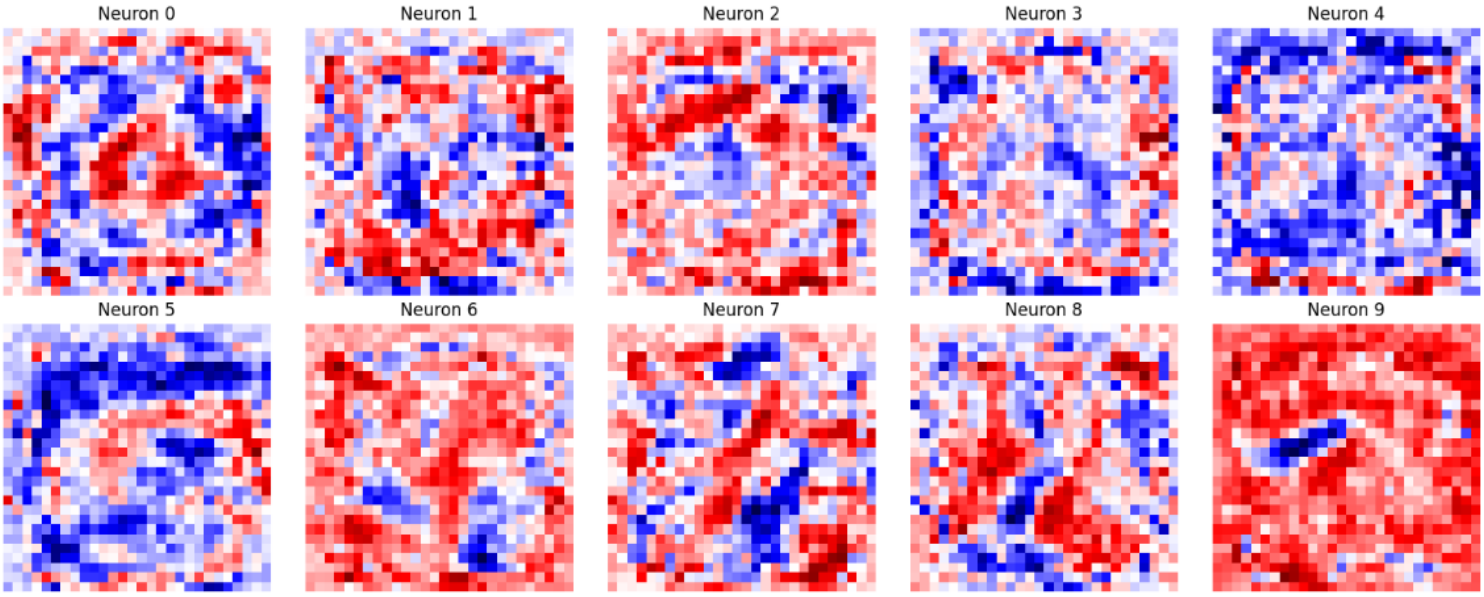
```
Epoch [1/15], Loss: 0.4419
Epoch [2/15], Loss: 0.1747
Epoch [3/15], Loss: 0.1184
Epoch [4/15], Loss: 0.0861
Epoch [5/15], Loss: 0.0655
Epoch [6/15], Loss: 0.0517
Epoch [7/15], Loss: 0.0418
Epoch [8/15], Loss: 0.0330
Epoch [9/15], Loss: 0.0265
Epoch [10/15], Loss: 0.0225
Epoch [11/15], Loss: 0.0179
Epoch [12/15], Loss: 0.0140
Epoch [13/15], Loss: 0.0100
Epoch [14/15], Loss: 0.0089
Epoch [15/15], Loss: 0.0078
Test Accuracy (Standard MNIST): 0.9794
```

Training loss decreased smoothly from 0.4419 to 0.0078 across 15 epochs, indicating stable convergence.

Final Test Accuracy: **97.94%**

This confirms that the FCNN is capable of learning discriminative representations for digit classification despite lacking explicit spatial awareness.

5. Weight Visualisation (First Hidden Layer Analysis)



The first hidden layer consists of 256 neurons, each receiving 784 input weights. These weights were reshaped into 28×28 matrices and visualised using heatmaps.

Observations:

- Some neurons show strong circular or curved weight patterns, suggesting activation for digits such as 0, 6, 8, and 9.
- Others highlight vertical stroke regions, corresponding to digits like 1.
- Certain neurons capture diagonal stroke patterns, associated with digits such as 4 or 7.
- A few neurons appear diffuse or noisy, indicating distributed feature contributions.

The red and blue regions in the heatmaps represent positive and negative weights, demonstrating which pixels contribute positively or negatively to neuron activation.
This analysis confirms that even without convolution, FCNNs learn primitive visual detectors in early layers.

6. Pixel Shuffle (Flattening) Experiment

Train new model on Shuffled Data:

```
Epoch [1/15], Loss: 0.4381
Epoch [2/15], Loss: 0.1728
Epoch [3/15], Loss: 0.1188
Epoch [4/15], Loss: 0.0880
Epoch [5/15], Loss: 0.0692
Epoch [6/15], Loss: 0.0542
Epoch [7/15], Loss: 0.0438
Epoch [8/15], Loss: 0.0356
Epoch [9/15], Loss: 0.0294
Epoch [10/15], Loss: 0.0239
Epoch [11/15], Loss: 0.0175
Epoch [12/15], Loss: 0.0156
Epoch [13/15], Loss: 0.0113
Epoch [14/15], Loss: 0.0104
Epoch [15/15], Loss: 0.0085
Test Accuracy (Shuffled MNIST): 0.9775
```

To evaluate the importance of spatial structure, a fixed random permutation of the 784 pixel positions was applied to all images in both training and test sets. The same network architecture and hyperparameters were used.

Results:

Dataset	Test Accuracy
Standard MNIST	97.94%
Shuffled MNIST	97.75%

The accuracy difference (~0.19%) is negligible.

7. Analysis & Interpretation

The nearly identical performance on shuffled MNIST reveals an important insight:
The Fully Connected Neural Network does not rely on spatial continuity of pixels. Instead, it treats the input as a flat vector of numbers and learns global statistical relationships between input positions and output labels.

Human vs Machine Perspective
If pixels are randomly scattered, a human observer would struggle to recognize digits because spatial shape information is destroyed. However, the FCNN continues to perform well because it does not explicitly use spatial relationships.
As long as the shuffle pattern is consistent across the dataset, the network can still learn the mapping between input indices and class labels.

8. Architectural Insight

This experiment highlights a fundamental limitation of Fully Connected Neural Networks:

- They do not preserve spatial locality.
- They lack local receptive fields.
- They do not use weight sharing.

Every pixel is treated independently and connected to all neurons in the next layer.
In contrast, Convolutional Neural Networks (CNNs):

- Exploit local spatial correlations.
- Use convolutional filters to detect patterns.
- Are translation-invariant.

Thus, while FCNNs perform well on simple image tasks like MNIST, they are not optimal for complex vision problems.

9. Key Findings from Latent Space Analysis

1. The FCNN achieves high classification accuracy on MNIST without explicit spatial modeling.
2. The first hidden layer learns interpretable primitive visual patterns.
3. Shuffling pixels does not significantly reduce performance.
4. This demonstrates that FCNNs are insensitive to spatial structure.
5. While effective for simple datasets, FCNNs are not ideal for complex vision tasks.

10. Conclusion

This experiment highlights a fundamental architectural limitation of Fully Connected Neural Networks in image processing tasks. Although they can achieve high accuracy on MNIST, they do not inherently capture spatial relationships between pixels.

The results justify the need for Convolutional Neural Networks (CNNs) in modern computer vision applications, where preserving spatial locality is crucial.

Part 3: Stress Testing & Robustness

Dataset: Tiny ImageNet 10 - a subset of tiny-imagenet-200 available at https://www.kaggle.com/datasets/akash2sharma/tiny-imagenet (use "tiny-imagenet.zip" provided with this question paper)

1. Problem Definition

The objective of this section is to analyze the training stability and robustness of deep Fully Connected Neural Networks (FCNNs) under different architectural and optimization settings.

Using a subset of Tiny ImageNet (10 classes, 64×64 RGB images), two major investigations were conducted:

1. **Vanishing Gradient Demonstration**
2. **Ablation Study on key training components**

The aim is to experimentally validate how activation functions, normalization techniques, learning rate, regularization, and optimizer choice influence deep network training behavior.

2. Dataset Description & Preprocessing

The Tiny ImageNet subset contains:

- 3,500 training images
- 500 validation images
- 10 object classes
- Image size: 64×64 RGB

Each image was:

1. Resized to 64×64 (if required)
2. Converted to tensor format
3. Normalized using mean = [0.5, 0.5, 0.5] and std = [0.5, 0.5, 0.5]
4. Flattened into a 12,288-dimensional vector before entering the FCNN

Batch size used: 64

Training device: CUDA-enabled GPU

3. Very Deep FCNN Architecture

A deep fully connected architecture (8+ layers) was implemented.

Architecture:
12288
→ 2048
→ 1024
→ 1024
→ 512
→ 512
→ 256
→ 128
→ 64
→ 10
Two configurations were tested:

- Experiment A: Sigmoid activations throughout
- Experiment B: ReLU activations + Batch Normalization + Dropout (0.3)

```
Epoch [1/8], Loss: 2.3130
Epoch [2/8], Loss: 2.3081
Epoch [3/8], Loss: 2.3059
Epoch [4/8], Loss: 2.3089
Epoch [5/8], Loss: 2.3071
Epoch [6/8], Loss: 2.3017
Epoch [7/8], Loss: 2.0969
Epoch [8/8], Loss: 2.0578
Validation Accuracy (Sigmoid): 0.168
```

```
Epoch [1/8], Loss: 2.1789
Epoch [2/8], Loss: 1.8916
Epoch [3/8], Loss: 1.7693
Epoch [4/8], Loss: 1.6847
Epoch [5/8], Loss: 1.6283
Epoch [6/8], Loss: 1.5591
Epoch [7/8], Loss: 1.4912
Epoch [8/8], Loss: 1.4614
Validation Accuracy (ReLU + BatchNorm): 0.414
```

Observations:

1. For Experiment A we can observe here that the loss is ~2.30 for many epochs and Validation accuracy is 16.8% and for 10 classes, random guess ≈ 10%, so this model is barely learned.
2. For Experiment B we can observe that the loss is steadily decreasing, the initial grad norm is ~30 and final grad norm is ~0.2 – 0.5 and the validation accuracy is 41.4%.
3. So for Sigmoid Gradients: 10^-6 → 10^-11 but for ReLU + BN Gradients: 30 → 0.3 is the magnitude. That is a difference of ~10 billion times larger gradient flow. This is exactly why deep sigmoid networks fail.

4. Vanishing Gradient Demonstration

To analyze gradient flow, the L2 norm of the first layer gradients was recorded during training.

Experiment A – Sigmoid Activation

- Validation Accuracy: 16.8%
- Gradient Norms: Decreased from ~1e-6 to ~1e-11

The extremely small gradient magnitudes indicate severe vanishing gradient behavior. Due to near-zero updates in early layers, the network failed to learn meaningful representations, resulting in near-random classification performance.

```
First 5 gradient norms: [1.5213781807688065e-06, 1.6338977957275347e-06, 1.5880725641181925e-06, 1.6473458117616246e-06, 1.5270
942412826116e-06]
Last 5 gradient norms: [9.566325509524631e-10, 4.1767105329881815e-09, 3.378284041399837e-10, 2.7254372980678454e-08, 2.4354917
388391684e-11]
```

Observations:

This is exponential decay of gradients, here the first layer is basically receiving almost zero updates. That is why loss stayed around 2.30 and accuracy remained near random (~60%). So we successfully demonstrated Vanishing gradients.

Experiment B – ReLU + Batch Normalisation

- Validation Accuracy: 41.4%
- Gradient Norms: Started around ~30 and stabilized between ~0.2–0.5

ReLU prevented saturation effects, while Batch Normalisation stabilized intermediate activations. This allowed effective gradient propagation and significantly improved performance.

```
First 5 gradient norms: [30.735515594482422, 6.741438865661621, 3.77988862991333, 2.2433767318725586, 1.7597990036010742]
Last 5 gradient norms: [0.21443580090999603, 0.3295880854129791, 0.49945953488349915, 0.31371748447418213, 0.5742082595825195]
```

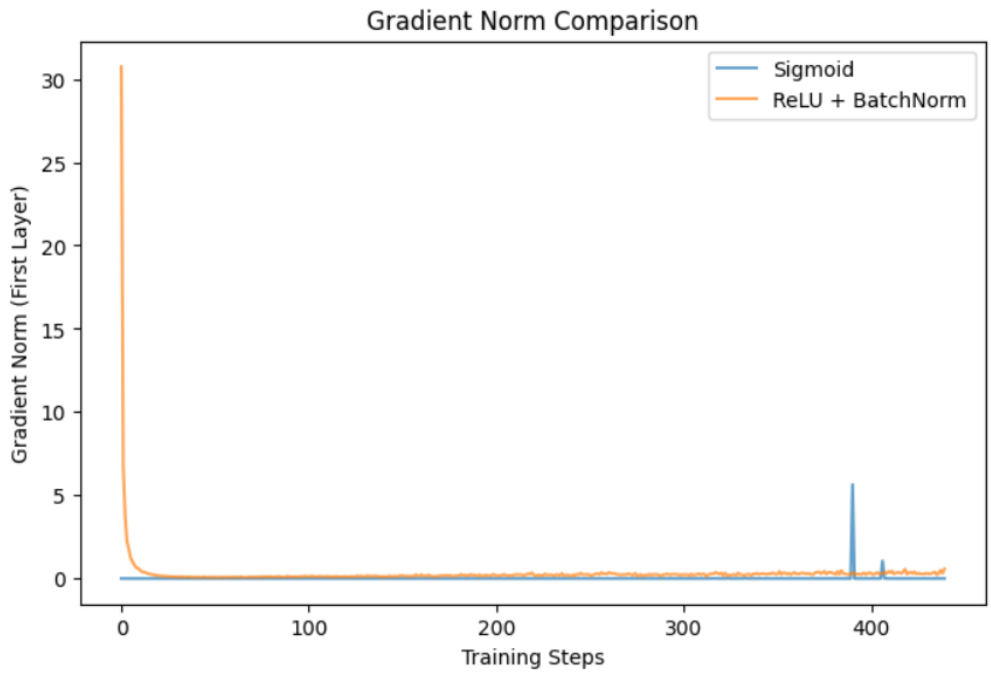
Comparative Analysis

Model	Validation Accuracy
Sigmoid (Deep)	16.8%
ReLU + BatchNorm	41.4%

The gradient norm plot clearly shows collapsing gradients in the sigmoid network, whereas the ReLU + BatchNorm network maintains healthy gradient magnitudes throughout training.

This experimentally validates the vanishing gradient problem and demonstrates modern solutions.

Plotted the "Gradient Norm" (magnitude of the weights' updates) for the first layer in both experiments.



Observations:

The gradient norm plot further validates the vanishing gradient phenomenon. The sigmoid-based network exhibits near-zero gradient magnitudes throughout training, indicating that error signals fail to propagate effectively to earlier layers. In contrast, the ReLU + Batch Normalisation network maintains significantly higher and stable gradient magnitudes. This enables continuous and meaningful updates to the first layer weights, resulting in substantially improved performance.

5. Ablation Study

To evaluate robustness, controlled modifications were applied to the ReLU + BatchNorm baseline.

Results Summary

Configuration	Validation Accuracy
Baseline (Adam, lr=0.001)	41.4%
No Dropout	41.2%
Learning Rate ×10 (0.01)	41.2%
Learning Rate ÷10 (0.0001)	33.2%
SGD Optimizer (0.001)	13.0%
Deep Sigmoid Network	16.8%

6. Analysis of Ablation Results

6.1 Removing Dropout

Training loss decreased faster, but validation accuracy remained almost unchanged. This indicates that dropout provided mild regularization but was not the primary performance driver in this setup.

```
Epoch [1/8], Loss: 1.8803
Epoch [2/8], Loss: 1.6457
Epoch [3/8], Loss: 1.5486
Epoch [4/8], Loss: 1.4302
Epoch [5/8], Loss: 1.3473
Epoch [6/8], Loss: 1.2728
Epoch [7/8], Loss: 1.1793
Epoch [8/8], Loss: 1.0579
Validation Accuracy (No Dropout): 0.412
```

Observations:

1. Without dropout, the training loss decreased faster compared to the baseline model.

2. However, validation accuracy remained nearly the same (41.2% vs 41.4%).

3. This suggests that dropout provides slight regularization benefits but is not the primary factor affecting performance in this setup.

4. The model does not drastically overfit within 8 epochs, but dropout contributes to improved stability.

6.2 Learning Rate Sensitivity

- Higher LR (0.01) did not cause divergence due to Adam's adaptive mechanism.
- Lower LR (0.0001) led to underfitting and significantly lower accuracy (33.2%).

This confirms that too small a learning rate slows convergence substantially.

```
Epoch [1/8], Loss: 2.0008
Epoch [2/8], Loss: 1.7878
Epoch [3/8], Loss: 1.7361
Epoch [4/8], Loss: 1.6750
Epoch [5/8], Loss: 1.6416
Epoch [6/8], Loss: 1.5824
Epoch [7/8], Loss: 1.5758
Epoch [8/8], Loss: 1.5305
Validation Accuracy (LR x10): 0.412
```

Observations:

1. Increasing the learning rate by a factor of 10 did not cause training instability.

2. Loss continued to decrease, although not significantly faster than baseline.

3. Validation accuracy remained around 41.2%, showing no improvement.

4. This demonstrates the robustness of the Adam optimizer, which adapts learning rates per parameter and prevents divergence even at higher global learning rates.

```
Epoch [1/8], Loss: 2.3622
Epoch [2/8], Loss: 2.2706
Epoch [3/8], Loss: 2.1745
Epoch [4/8], Loss: 2.0854
Epoch [5/8], Loss: 2.0129
Epoch [6/8], Loss: 1.9543
Epoch [7/8], Loss: 1.8987
Epoch [8/8], Loss: 1.8582
Validation Accuracy (LR ÷10): 0.332
```

Observations:

1. Reducing the learning rate significantly slowed down training.

2. The loss decreased very gradually and remained relatively high after 8 epochs.

3. Validation accuracy dropped to 34.6%, indicating underfitting.

4. This confirms that too small a learning rate prevents effective convergence within limited training epochs.

6.3 Optimizer Comparison (Adam vs SGD)

Switching from Adam to vanilla SGD resulted in dramatic performance degradation (13% accuracy).

```
Epoch [1/8], Loss: 2.4155
Epoch [2/8], Loss: 2.3952
Epoch [3/8], Loss: 2.3840
Epoch [4/8], Loss: 2.3781
Epoch [5/8], Loss: 2.3558
Epoch [6/8], Loss: 2.3497
Epoch [7/8], Loss: 2.3455
Epoch [8/8], Loss: 2.3275
Validation Accuracy (SGD): 0.13
```

Observations:

SGD (lr = 0.001):

- Loss barely decreased (2.41 → 2.32)
- Validation Accuracy = **13%**
- Almost random performance (10 classes → 10% baseline)

This clearly shows:

Adam >> SGD for this deep architecture under same learning rate.

Loss barely decreased, indicating poor convergence.

This demonstrates the importance of adaptive optimizers in training deep networks efficiently.

Ablation Study

To evaluate model robustness, controlled modifications were applied to the ReLU + BatchNorm baseline architecture.

Results Summary:

Configuration	Validation Accuracy
Baseline (Adam, lr=0.001)	41.4%
No Dropout	41.2%
Learning Rate ×10	41.2%
Learning Rate ÷10	34.6%
SGD Optimizer	13.0%
Deep Sigmoid Network	16.8%

7. Key Findings

1. Removing dropout had minimal impact on validation accuracy, suggesting that regularization was not the dominant factor in this setup.

2. Increasing the learning rate by 10x did not destabilize training significantly due to Adam's adaptive behavior.

3. Reducing the learning rate by 10x resulted in underfitting and lower validation accuracy (34.6%).

4. Switching from Adam to vanilla SGD caused the largest performance drop (13%), indicating that optimizer choice plays a crucial role in deep network training.

5. The sigmoid-based deep network also performed poorly (16.8%) due to vanishing gradients.

6. Among all factors tested, optimizer choice and activation function had the most significant impact on performance.

8. Conclusion

This section experimentally demonstrates core deep learning principles:

- The vanishing gradient problem in deep sigmoid networks
- The effectiveness of ReLU and Batch Normalisation
- The critical role of optimizer choice
- The sensitivity of learning rate

The results confirm that architectural design and optimization strategies are crucial for stable deep network training.